

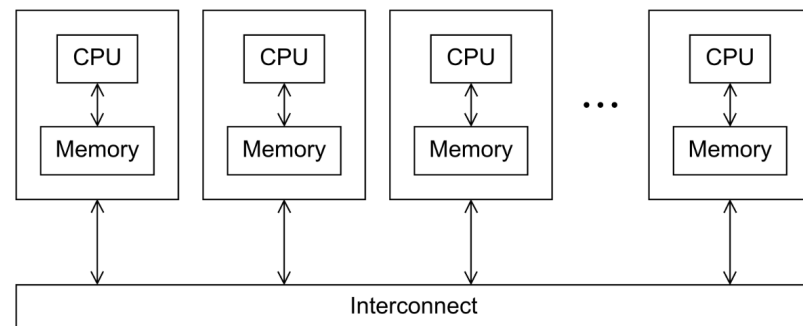
MSDM 5001

An Introduction to Computational and Modelling Tools

Lecture 7: Distributed-Memory Parallel Computing

Distributed memory computing

- Workers with separate address spaces (typically across machines)
 - Archetypal example: computer cluster
- Terminology
 - Each worker/machine is called a “node”
 - Each program is called a process
- More scalable compared to shared-memory computing
 - Why?



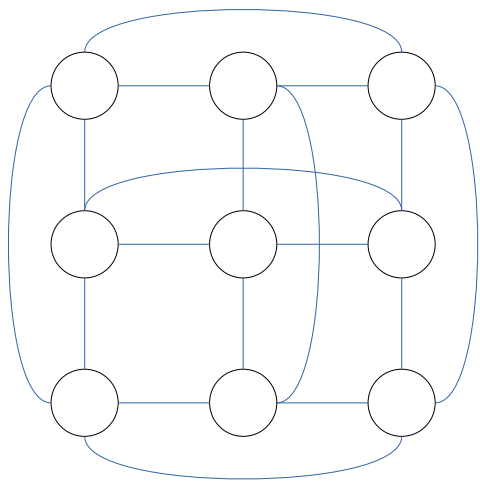
Communication hardware in distributed computing

- Communication speed is often main bottleneck in distributed systems
- Common connections
 - Ethernet
 - Most common, often found in enterprise data centers, cloud providers, and most cluster computing
 - Infiniband
 - Low latency, high throughput
 - Most often found in HPC and/or AI training
 - For tightly coupled parallel tasks

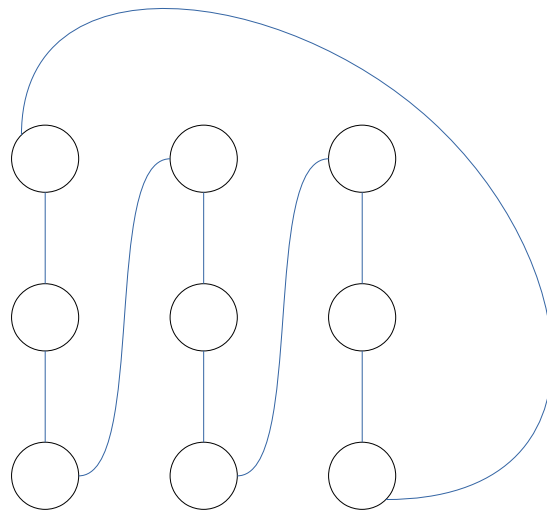


Communication hardware in distributed computing

- Different ways to connect machines, i.e. network topology
- Which is “better”?



“toroidal mesh”



“ring”

Communication hardware in distributed computing

- A way to measure “connectivity” of a network: bisection bandwidth
 - Minimum number of cuts needed to divide a network in two halves
- Assume we have $p = n^2$ nodes in a network
 - What is the bisection bandwidth of a ring?
 - What is the bisection bandwidth of a toroidal mesh? (assume even n)
 - Is it possible to have even higher bisection bandwidth? What is the theoretical maximum? Is this maximum practical?
 - Can you devise a network with bisection bandwidth linear in p ?

Communication software in distributed computing

- Most common paradigm: message passing
- The “de facto standard”: Message Passing Interface (MPI)
 - API to send and receive messages (along with other things)
 - Physical connection abstracted away
 - Different implementations, e.g. **OpenMPI**, MPICH
 - We will be using Python bindings provided by `mpi4py`



Installing and setting up mpi4py

1. Install an MPI implementation

- Refer to “OpenMPI_Installation.pdf” on Canvas

2. Install mpi4py

```
uv pip install mpi4py
```

3. Run an MPI program (for later)

```
mpiexec -n 4 python script.py
```

MAKE SURE YOU ARE IN THE VENV WHERE YOU INSTALLED mpi4py!!!!

Most basic form of an MPI program

- Communicator
 - Defines a group of process that can communicate with each other
 - Can define your own topology
- Rank
 - Number assigned to each process
- Size
 - Number of processes

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# your code here
```


Example: a “hello world” mpi program

- Run the following with `mpirun`

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

print(f"Hello from rank {rank} out of {size} processes")
```

Point-to-point communication

- `comm.send(obj, dest, tag=0)`
 - Send picklable python objects
 - “Blocking” (see later)
 - Tag is for organizing messages
- `comm.recv(source, tag=0, status=None)`
 - status: a container for metadata about the message received (e.g. who sent it, what tag it has, error info)

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    # Master process code
    data = something
    comm.send(data, dest=1)
else:
    # Worker process code
    data = comm.recv(source=0)
    # Process data
```

master-worker pattern
SPMD!

Uppercase vs Lowercase

- Lowercase communication (send, recv)
 - For general picklable python objects
 - Convenient, general-purpose, but slow
- Uppercase communication (Send, Recv)
 - For buffer-like objects with contiguous memory layout (e.g. Numpy arrays)
 - Fast and efficient, zero-copy possible
 - In HPC applications, this is most common
 - `Recv(buf, source=None, tag=0)`



pre-allocated buffer

Blocking vs Non-blocking

- `Send()` and `Recv()` are blocking
 - Only returns when either (i) data have been copied to system buffer, or (ii) the receiver has received the data (depends on message and MPI implementation)
 - Think of system buffer as a post office keeping your package; compare this to you sending package to receiver directly
- `Isend()` and `Irecv()` are non-blocking
 - Returns a “request” object immediately, only initiate the transfer
 - Doesn’t waste time waiting
 - Must be used with the `Wait()` method of the request object before actually using the data

An example

```
if rank == 0:
    comm.send("message_from_0", dest=1)
    data = comm.recv(source=1)

elif rank == 1:
    comm.send("message_from_1", dest=0)
    data = comm.recv(source=0)
```

Anything wrong?

```
if rank == 0:
    req = comm.isend("message_from_0", dest=1)
    data = comm.recv(source=1)
    req.wait() # Ensure send completes

elif rank == 1:
    req = comm.isend("message_from_1", dest=0)
    data = comm.recv(source=0)
    req.wait() # Ensure send completes
```

Exercise 1 – Ring Communication

- Create an MPI program in which
 - Each process sends its rank number to the next process
 - Process 0 starts with the value 0
 - Each subsequent process adds its own rank to the received value
 - The final accumulated sum should end up back at process 0
 - Process 0 should print the final sum
- Example with 4 processes
 - Process 0: sends 0 to process 1
 - Process 1: receives 0, adds 1, sends 1 to process 2
 - Process 2: receives 1, adds 2, sends 3 to process 3
 - Process 3: receives 3, adds 3, sends 6 back to process 0
 - Process 0: receives 6 and prints it
- Complete and fill in the missing code in Exercise_1_Blank.py

Exercise 2 – Pairwise Swapping

- Create an MPI program in which
 - Even-ranked processes are paired with odd-ranked processes (0 with 1, 2 with 3, etc.)
 - Each pair exchanges data: even rank sends array A and receives array B, odd rank sends array B and receives array A
- Complete and fill in the missing code in Exercise_2_Blank.py

Collective communication

- Involve ALL processes in a communicator
- Examples:
 - Bcast: broadcasting, one-to-all
 - Scatter: one-to-all, splits data and send to other processes
 - Gather: all-to-one, collects data from all processes
 - Reduce: all-to-one, combines data from all processes using an aggregate operation, e.g. min, max, mean
 - Allgather, all-to-all, like Gather, but all processes receive
 - Allreduce: all-to-all, like Reduce, but all processes receive
 - Barrier: similar to barrier we discussed in shared-memory system

Collective communication

- `comm.Bcast(buffer, root=root)`

```
# Create data on all processes
if rank == 0:
    data = np.array([1.0, 2.0, 3.0, 4.0])
else:
    data = np.empty(4, dtype=np.float64)

# Broadcast from rank 0 to all other processes
# Must be called by ALL processes
comm.Bcast(data, root=0)

print(f"Rank {rank}: {data}")
```

Collective communication

- `comm.Scatter(send_buffer, recv_buffer, root=root)`

```
elements_per_process = 3

# Root prepares data for all processes
if rank == 0:
    # Total: size * elements_per_process elements
    send_data = np.arange(size * elements_per_process, dtype=np.int32)
    print(f"Root has data: {send_data}")
else:
    send_data = None

# Each process receives elements_per_process elements
recv_data = np.empty(elements_per_process, dtype=np.int32)

comm.Scatter(send_data, recv_data, root=0)

print(f"Rank {rank} received: {recv_data}")
```

Collective communication

- `comm.Gather(send_buffer, recv_buffer, root=root)`

```
# Each process has its own data
send_data = np.array([rank], dtype=np.int32)

# Root prepares buffer to receive from all processes
if rank == 0:
    recv_data = np.empty(size, dtype=np.int32)
else:
    recv_data = None

# ordering is by rank number
comm.Gather(send_data, recv_data, root=0)

if rank == 0:
    print(f"Gathered data: {recv_data}")
```

Collective communication

- `comm.Reduce(send_buffer, recv_buffer, op=op, root=root)`

```
# Each process has its own value
send_data = np.array([rank], dtype=np.int32)

# Root prepares buffer for result
if rank == 0:
    recv_data = np.empty(1, dtype=np.int32)
else:
    recv_data = None

comm.Reduce(send_data, recv_data, op=MPI.SUM, root=0)

if rank == 0:
    print(f"Sum of all ranks: {recv_data[0]}")
```

- `MPI.SUM`
- `MPI.MAX`
- `MPI.MIN`
- `MPI.PROD`
- `MPI.LAND`
- `MPI.LOR`

Exercise 3 – Parallel Matrix-Vector Multiplication

- In a lot of applications, we are interested in finding the largest eigenvalue of a matrix (examples?)
- One algorithm is “power iteration”:

Complexity? →

```
Input: n x n Matrix A, initial vector  $x_0$  ( $n \times 1$ ), tolerance  $\epsilon$ , max_iterations
Output: Dominant eigenvalue  $\lambda$  and eigenvector  $x$ 

1. Initialize:
    $x = x_0$ 
    $x = x / ||x||$  // Normalize initial vector

2. For k = 1 to max_iterations:
    $y = A * x$ 
    $\lambda = x^T * y$ 
    $x_{\text{new}} = y / ||y||$ 
   If  $||x_{\text{new}} - x|| < \epsilon$ : // Check convergence
       return ( $\lambda$ ,  $x_{\text{new}}$ )
    $x = x_{\text{new}}$  // Update for next iteration

3. return ( $\lambda$ ,  $x$ ) // Return after max iterations
```

Exercise 3 – Parallel Matrix-Vector Multiplication

- Parallelize matrix-vector multiplication by breaking up the matrix
- Set up the relevant data in the root node
- Your algorithm should consist of 3 phases:
 - Breaking up the matrix and sending it to different processes
 - Each process compute part of the result using the partitioned matrix
 - Combine results from different processes

Running MPI programs on a cluster

- Your code stays exactly the same
- Extra complexity: workload manager (e.g. SLURM)
 - Write a job submission bash script which specifies, among others, number of nodes, how many MPI processes per node, time limit
 - Submit the job script and monitor it
 - We won't go into details here

Useful references/resources

- Section 2.3.4 and Chapter 3 from “An Introduction to Parallel Programming” by Peter Pacheco
- mpi4py official documentation page:
<https://mpi4py.readthedocs.io/en/stable/>